Sprint 3 — Relatório de refatoração

Projeto P07 — Controle de Frequência · Branch `sprint3/testes` · Atualizado em 09/11/2026

1. Contexto e escolha dos itens

O inventário (`docs/qualidade/divida-tecnica.md`) tem nove itens: oito identificados por leitura manual apoiada por ChatGPT em 08/11 e um (DT-09) proposto por ferramenta em 09/11. Foram refatorados três, em commits separados e com o ID do item:

Item	Por que foi escolhido (uma frase)
DT-02 (`FOR UPDATE` repetido)	Trecho dentro do módulo mais coberto (100% de linhas do serviço) e correção mecânica, de baixo risco.
DT-04 (relógio espalhado)	Quatro leituras diretas do relógio impediam testes determinísticos; centralizá-las dá um único ponto para controlar o tempo.
DT-01 (`registrar` longo)	Era o método mais longo e que mais misturava responsabilidades do serviço (55 linhas pelo ESLint).

Em todos os casos as suítes foram rodadas **antes e depois** (logs em `docs/qualidade/evidencias/suites/`): a **v1 oficial** com 117 testes e a **v2** com 48. Nenhum teste quebrou em nenhum estado, nem no código original (117/117 e 48/48).

2. DT-02 — SQL de bloqueio duplicado · commit `aca6e77`

Antes (3 cópias, em `abrir`, `encerrar` e `registrar`):

```
        const aulas = await tx.$queryRaw<{ id: string }[]>`SELECT "id" FROM "aulas" WHERE "id" =
${aulaId}::uuid FOR UPDATE`;
```

Depois: `private async bloquearAula(tx, aulaId)` e `const aulas = await this.bloquearAula(tx, aulaId);` nos três métodos.

- **Refatoração:** Extract Function. **Melhora:** *Duplicated Code* e consistência; a regra de bloqueio passa a ter um único lugar para mudar.
- **Testes:** v1 117/117 → 117/117; v2 48/48 → 48/48.
- **Ferramenta:** o jscpd apontava um clone de 9 linhas entre `abrir` e `encerrar` no original; ele some no limiar padrão. Com limiar sensível, restam 7 linhas repetidas (a verificação `if (aulas.length === 0) throw ...`), registradas como **DT-09**.

3. DT-04 — relógio espalhado · commit `95779b6`

Antes: `Date.now()` em `estaAtiva` e `new Date()` em `abrir`, `encerrar` e `listarAtivas`. **Depois:** `private agora(): Date { return new Date(); }` e todas as leituras viram `this.agora()`.

- **Refatoração:** Extract Function (encapsular o relógio). **Melhora:** baixo acoplamento ao tempo do sistema e testabilidade; a v1 oficial controla o instante com um relógio falso (`vi.setSystemTime`) e testa a fronteira exata da expiração.
- **Testes:** v1 117/117 → 117/117; v2 48/48 → 48/48.
- **Ferramenta:** nenhuma regra do ESLint ou do jscpd detecta esse problema; ele só apareceu na leitura.

4. DT-01 — responsabilidades do `registrar` · commit `e5136f6`

Antes: um único método (~60 linhas) fazia a busca da chamada, a verificação da matrícula, a transação de tentativas e a tradução do resultado em erros HTTP. **Depois:**

```
        const { aulaId, alunoId } = await this.buscarMatricula(chamadaId, email);
        const resultado = await this.prisma.$transaction(/* tentativas e presença */);
        return this.interpretarRegistro(resultado, aulaId);
```

- **Refatoração:** Extract Method (`buscarMatricula` e `interpretarRegistro`). **Melhora:** *Long Method*, coesão e SRP; a transação continua inteira em `registrar`, preservando o bloqueio e a gravação das tentativas inválidas.
- **Testes:** v1 117/117 → 117/117; v2 48/48 → 48/48.
- **Ferramenta:** o ESLint (`max-lines-per-function`, limite 50) apontava `registrar` com **55 linhas** no original; no código atual são **31** e ele deixou de ser apontado.

5. Comparação das formas de identificação

- **Só a leitura manual/LLM encontrou:** DT-04 (relógio), DT-05 (normalização duplicada) e o acoplamento do serviço ao Prisma (DT-03). O madge apenas lista a dependência de `database/prisma.service` e não a classifica como problema. A leitura também explicou por que `FOR UPDATE` **não** é defeito (só a duplicação o é), enquanto uma regra automática não distingue isso.
- **Só a ferramenta quantificou:** o tamanho e a complexidade de `registrar` (55 linhas) e de `ChamadaCodigoProfessor` (108 linhas, complexidade 16; DT-07), e o resíduo de duplicação da DT-09, que a leitura não havia registrado. As ferramentas também permitiram **medir o efeito das refatorações** (55 → 31 linhas; clone de 9 linhas eliminado).
- **Ruído das ferramentas:** o `no-magic-numbers` deu 29 achados, boa parte irrelevante (status `204`, o `1000` e o `10` do arredondamento, a lista de opções de duração). O jscpd não achou a duplicação entre os dois componentes React (DT-08), porque o código é parecido mas não idêntico.
- **LLM versus ferramenta:** violações de SOLID e de *information hiding* (por exemplo, as responsabilidades misturadas de `registrar`) vieram da leitura apoiada por LLM; as regras de complexidade só pegaram o sintoma de tamanho.

6. Limites

Os testes usam mocks do Prisma e não provam concorrência real, rollback ou o comportamento do `FOR UPDATE` no PostgreSQL; a passagem das suítes é evidência de regressão dentro do escopo testado, não prova de equivalência. O build (`nest build`) não foi reexecutado em 09/11; valem os logs de 08/11. Seis dívidas seguem abertas (DT-03, DT-05, DT-06, DT-07, DT-08 e DT-09). A classificação dos achados de ferramenta foi feita pelo Claude e deve ser revisada pela equipe.